An Internship Report
Of
CodeAlpha Full Stack Development Internship

Submitted
To
Department of Computer Science and Engineering
IILM UNIVERSITY

In partial fulfilment of the requirement of degree of
B.Tech (Computer Science and Engineering)

By

Name of Student: PRABHAT KUMAR JHA
Roll Number of Student: 25SCS1003004567
Section: 2CSE36
Batch: 2025-2029

CANDIDATE'S DECLARATION

I, PRABHAT KUMAR JHA, do hereby declare that the internship report titled "CodeAlpha Full Stack Development Internship" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own and has not been submitted to any other institute for any degree/diploma requirement, and further I shall be solely responsible for any kind of copyright violation in this regard.

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluably constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to thank the CodeAlpha team for providing me the opportunity and the structured, task-based platform to learn and gain hands-on exposure in the domain of Full Stack Development. The freedom to design and build two complete web applications from scratch, and the requirement to document and publish them, taught me far more than a purely theoretical exercise would have. I am also thankful to my institute, IILM UNIVERSITY, and the faculty of the Department of Computer Science and Engineering for their continuous support and encouragement.

I express my gratitude to my parents for their blessings.

TABLE OF CONTENTS

3. INTERNSHIP COMPLETION CERTIFICATE

OFFER LETTER

CODE ALPHA

YOUR DREAM OUR PASSION

INTERNSHIP OFFER LETTER

7th July 2026 **STUDENT ID: CA/DF1/194729**

Name: Prabhat Kumar Jha

Welcome to CodeAlpha

Congratulations! We are delighted to make you an offer as **Full Stack Development internship** offer letter is to confirm your selection **Full Stack Development internship** at **CodeAlpha** and welcome you for the same effective from 10th July 2026 to 10th August 2026. Hope you will be doing well.

This Internship is observed by CodeAlpha as being a learning opportunity for you.

Your internship will embrace orientation and give emphasis on learning new skills with a deeper understanding of concepts through hands-on application of the knowledge you gained as an intern. You will acknowledge your obligation to perform all work allocated to you to the best of your ability within lawful and reasonable direction given to you.

We look forward to a worthwhile and fruitful association which will make you equipped for future projects.

Wishing you the most enjoyable and truly meaningful internship program experience.

Sincerely,

SwatiSri

Swati Srivastava
Founder & CEO

CODEALPHA SOFTWARE DEVELOPMENT

MSME

Phone +91 9336576683

Email services@codealpha.tech

Address Lucknow U.P

4. PROJECT DESCRIPTION

4.1 Introduction

This report presents a detailed account of the one-month Full Stack Development Internship undertaken at CodeAlpha. The internship was carried out remotely from 10th July 2026 to 10th August 2026 under Student ID CA/DF1/194729, as part of the requirement of the Bachelor of Technology (Computer Science and Engineering) programme at IILM UNIVERSITY. The offer letter confirming selection was issued on 7th July 2026 and the Certificate of Completion was issued on 25th August 2026.

CodeAlpha follows a task-based internship model: instead of passive lecture content, the intern is assigned a list of real development tasks and is expected to design, build, document and publish complete working applications within the internship period. Two tasks were selected and completed end to end during this internship:

- **Task 1 – Simple E-Commerce Store (project name ShopKart):** a transactional shopping application with a product catalogue, search and category filtering, user authentication, a persistent shopping cart, a stock-validated checkout and an order history.
- **Task 2 – Social Media Platform (project name CampusLoop):** a campus-oriented social network with authentication, a categorised post feed, likes, threaded comments, follow/unfollow relationships and live search.

Both applications were built on the same foundation—a Node.js and Express.js backend exposing a JSON REST API, a relational SQLite database accessed through prepared statements, session-based authentication with hashed passwords, and a hand-written HTML5/CSS3 vanilla JavaScript frontend served by the same Express process. No frontend framework and no build step were used, which was a deliberate choice: writing the DOM manipulation, state handling and API layer by hand made the underlying mechanics of a full-stack application far more visible than a framework would have allowed. Both projects were version-controlled with Git, published on GitHub and documented with a README file, as required by the internship guidelines.

4.2 Organization Profile

CodeAlpha is a technology organisation that runs structured virtual internship programs for engineering and computer-science students across domains such as Full Stack Development, Web Development, Java Programming, Python Programming and Data Science. Each selected candidate receives a formal offer letter carrying a unique Student ID, a defined internship window, and a task list from which the intern completes a required number of tasks.

As stated in the offer letter, "this internship is observed by CodeAlpha as being a learning opportunity" and it "will embrace orientation and give emphasis on learning new skills with a deeper understanding of concepts through hands-on application of the knowledge gained as an intern." The program is therefore project-driven rather than lecture-driven: assessment is based on the working applications the intern submits, their source code, and their documentation. On satisfactory completion, CodeAlpha issues a Certificate of Completion referencing the intern's Student ID and the internship period, which can be independently verified.

4.3 Problem Statement

A large number of engineering students learn the individual pieces of web development—HTML and CSS for structure and styling, JavaScript for interactivity, and SQL for data storage—as separate academic subjects. What is rarely practised is the part that actually defines full stack development: making those pieces cooperate. Very few students get structured exposure to how a browser request travels through a routing layer, how a session is established and carried on subsequent requests, how a password is stored safely, how related tables are designed so that data stays consistent, or how an operation that touches several tables at once is made safe against partial failure.

The problem addressed by this internship was therefore to move from isolated knowledge to an integrated, working system—to design and implement two complete full-stack web applications from an empty folder to a running, documented product. The two tasks were chosen deliberately because they represent the two dominant categories of real web applications and stress different engineering concerns:

- A transactional application (e-commerce), where correctness is critical: stock must not go negative, an order must never be half-written, and the price recorded against an order must not change when the catalogue is later re-priced.
- A social / content-driven application, where relationships and aggregation dominate: many-to-many links between users and posts, between users and other users, and per-viewer derived values such as "have I already liked this post" that must be computed efficiently rather than with one query per row.

4.4 Project Objectives

- To design and implement a complete full-stack architecture in which a single Node.js and Express.js process both exposes a versioned JSON REST API under /api and serves the static frontend, removing cross-origin complexity in development.
- To implement secure user registration and login using bcryptjs password hashing with a per-user salt, and server-side sessions with an HttpOnly cookie, so that no password and no user data ever travel in or are stored in the browser.
- To design normalised relational schemas in SQLite with primary keys, unique constraints, foreign keys and ON DELETE CASCADE rules, and to enforce them by explicitly enabling PRAGMA foreign_keys.
- To use parameterised prepared statements for every database access so that the applications are structurally immune to SQL injection.
- To build, for Task 1, a product catalogue with keyword search and category filtering, a database-backed shopping cart that survives logout and device changes, and an atomic checkout that validates stock, records the order with a historical price snapshot, decrements inventory and clears the cart inside a single database transaction.
- To build, for Task 2, a categorised social feed with likes, lazily loaded comment threads, follow and unfollow relationships, anonymous and "help needed" post flags, and instant client-side search.
- To compute per-post aggregates (like counts, comment counts, and per-viewer state) in a single SQL statement using correlated subqueries, avoiding the N+1 query problem.

- To develop a clean, consistent and accessible user interface using only HTML5, CSS3 (custom properties, Flexbox and Grid) and vanilla JavaScript, including loading states, empty states and inline error reporting.
- To use Git and GitHub for version control and to document each project with a README covering features, setup and API reference, as required for submission.

4.5 Scope of the Project

The scope of this internship spans the complete development lifecycle of two web applications: requirement analysis, database design, backend API design and implementation, authentication and authorisation, frontend implementation, manual functional testing, documentation and publication to GitHub. Every layer of both applications—database schema, server routes, middleware, client-side logic and stylesheet—was written from scratch during the internship period.

The following were consciously kept outside the scope, as they were not required by the task specification: integration with a live payment gateway (checkout records the order and shipping details but does not process a real payment); an administrative back-office for managing the catalogue, for which a transactional seed script is used instead; file and image uploads, with user avatars generated from username initials rather than stored images; e-mail delivery, password reset and two-factor authentication; and deployment to a cloud provider, since both applications run against a local file-based SQLite database. The applications were developed and verified in a local development environment on http://localhost:5000.

4.6 Technologies and Tools Used

- **Frontend:** HTML5 (semantic markup, responsive viewport meta tag), CSS3 (custom properties for design tokens, Flexbox, CSS Grid, keyframe animations, prefers-reduced-motion media query), and vanilla JavaScript (ES6+, Fetch API, async/await, template literals, event delegation, URLSearchParams).
- **Backend runtime and framework:** Node.js with Express.js—Express 4.x in Task 1 and Express 5.x in Task 2—using CommonJS modules.
- **Database:** SQLite accessed through the better-sqlite3 driver (synchronous, in-process, no ORM), with schema created idempotently via CREATE TABLE IF NOT EXISTS.
- **Authentication and security:** bcryptjs for password hashing (cost factor 10) and express-session for server-side session management with an HttpOnly connect.sid cookie.
- **Supporting libraries:** dotenv for environment configuration (port and session secret kept out of source control) and cors for cross-origin headers.
- **Development tools:** Visual Studio Code with the Prettier formatter and Live Server extension, Git and GitHub for version control, npm for dependency management, Chrome DevTools for network and DOM inspection, and DB Browser for SQLite for verifying data.

A short comparison of the two project stacks is given below.

Layer	Task 1 - ShopKart (E-Commerce)	Task 2 - CampusLoop (Social)
Server framework	Express 4.x	Express 5.x
Database driver	better-sqlite3 11.x	better-sqlite3 13.x
Database file	server/data/store.db	server/data/campusloop.db
Password hashing	bcryptjs 2.x, cost 10	bcryptjs 3.x, cost 10
Session lifetime	7 days	1 day
Tables	users, products, cart_items, orders, order_items	users, posts, comments, likes, follows
Route organisation	Modular routers in server/routes/	Routes defined inline in server.js
Frontend pages	6 HTML pages	2 HTML pages

4.7 System Architecture

Both applications follow the same layered, single-process architecture. Express is configured so that API routes are matched first, static assets are served next, and any remaining path falls back to the frontend entry page. Because one process serves both the API and the client, the browser and the server share an origin, so the session cookie is sent automatically and no cross-origin configuration is needed during development.

Layer	Responsibility and implementation
Presentation Layer	HTML pages styled by a single stylesheet, with all dynamic content injected by vanilla JavaScript. Every network call goes through one shared wrapper around the Fetch API that sends credentials: "include", parses JSON defensively and converts a non-2xx response into a JavaScript error carrying the server's message.
Static Delivery Layer	express.static() serves the public/ directory; a catch-all route returns the entry page for unknown non-API paths.
API/Routing Layer	Stateless JSON endpoints grouped by resource under /api. Task 1 uses four mounted Express routers (auth, products, cart, orders); Task 2 defines its twelve endpoints inline. Every response is JSON with a uniform { error: "..." } envelope on failure.
Middleware Layer	express.json() for body parsing, cors() for headers, express-session for session handling, and an authentication guard that rejects unauthenticated requests with HTTP 401 before the route handler executes.
Data Access Layer	Prepared statements created with db.prepare() and bound positionally or by name. Multi-table writes are wrapped in db.transaction() so that they either fully commit or fully roll back.
Persistence Layer	A single SQLite file per project holding five tables, with foreign keys and cascade rules enforced by PRAGMA foreign_keys = ON.

The request path for a typical authenticated action is therefore: browser event → shared fetch wrapper → Express router → session middleware resolves the signed cookie into a user id → authentication guard → route handler → prepared statement (inside a transaction where required) → JSON response → the client re-renders the affected region of the page.

4.8 Methodology

The internship ran for four weeks, from 10th July 2026 to 10th August 2026. An incremental, iterative methodology was followed: for each task the database schema was designed first, then the API was built and verified endpoint by endpoint, and only then was the interface layered on top. This ordering meant that by the time any screen was written, the data it needed was already available and tested. The week-wise plan and its mapping to the two tasks is summarised below.

Week	Dates (2026)	Task	Work carried out
1	10 Jul - 16 Jul	Task 1 - backend	Environment setup (Node.js, npm, Git, VS Code). Requirement analysis for the e-commerce store. Design of the five-table SQLite schema with foreign keys and the UNIQUE (user_id, product_id) constraint on the cart. Implementation of the registration, login, logout and session-probe endpoints with bcrypt hashing. Product listing, search, category and detail endpoints. A transactional seed script loading 21 products across 5 categories.
2	17 Jul - 24 Jul	Task 1 - frontend and checkout	Cart endpoints with per-user ownership scoping. The atomic checkout endpoint: stock pre-validation, order and order-item insertion with a price snapshot, inventory decrement and cart clearing inside one transaction. Order history endpoints. All six frontend pages, the shared API and navbar module, debounced search, skeleton loading states and the responsive stylesheet. README, Git history and GitHub publication.
3	25 Jul - 2 Aug	Task 2 - backend	Requirement analysis for the social platform. Design of the CampusLoop schema, including the two composite-key join tables (likes and follows) that make duplicate likes and duplicate follows impossible at the database level. Authentication endpoints. The feed query computing like count, comment count and three per-viewer flags in a single statement. Post creation and ownership-checked deletion, like toggle, comment endpoints and the follow toggle with a self-follow guard.
4	3 Aug - 10 Aug	Task 2 - frontend and submission	The three-column feed interface: navbar with live search, profile widget, post composer with category and flag controls, post cards and collapsible comment threads. A single delegated click handler covering five

			distinct card actions. Client-side search across content, category and author. Manual functional testing of both applications end to end, documentation, and final submission of both projects.

Note: As stated in the offer letter, the internship was observed by CodeAlpha as a learning opportunity, and no stipend was associated with it.

Each feature was verified manually as it was completed, using the browser for the happy path and Chrome DevTools together with direct API calls for error paths—for example confirming that an unauthenticated request to a protected endpoint returns HTTP 401, that requesting another user's cart line or order returns 404 rather than disclosing its existence, that checking out with a quantity greater than the available stock is rejected with a specific message naming the product, and that attempting to delete another user's post is rejected with HTTP 403. Commits were made incrementally so that each working feature was recorded in version control.

4.9 Task 1 - Simple E-Commerce Store (ShopKart)

Overview. ShopKart is a complete online store in which one Express server on port 5000 exposes the REST API and serves the frontend. A visitor can browse a seeded catalogue of 21 products, search by keyword, filter by category and open a product page showing the description, price and live stock. After registering or logging in, the user can add items to a cart that is stored in the database rather than the browser, adjust quantities, remove lines and check out by entering a shipping name and address. Checkout validates stock, writes the order, decrements inventory and empties the cart as one indivisible operation, after which the order appears in the user's order history.

4.9.1 Key Features Implemented

- Product catalogue with keyword search and category filtering, plus an individual product page showing the description, price and current stock.
- User accounts with registration and login, passwords stored only as bcrypt hashes, and the signed-in state held in a server-side session.
- Database-backed shopping cart in which quantities can be changed and lines removed, and which survives logout and a change of device.
- Checkout that validates the requested quantities against available stock, records the order, reduces the inventory and empties the cart in a single transaction.
- Order history listing every past order with its line items, prices and shipping details.
- Responsive interface with loading placeholders, empty states, inline error messages and a live cart count in the navigation bar.

4.9.2 Database Design

The schema is created idempotently on start-up and consists of five tables. Foreign key enforcement is switched on explicitly, because SQLite leaves it off by default.

Table	Columns	Purpose and relationships
users	id, name, email (unique), password_hash, created_at	Account records. The plain password is never stored.
products	id, name, description, price, category, image_url, stock	Catalogue. Seeded with 21 products across Electronics, Clothing, Home, Books and Accessories.
cart_items	id, user_id, product_id, quantity	One row per user per product, enforced by UNIQUE (user_id, product_id). Cascades on deletion of the user or the product.
orders	id, user_id, total_amount, status, shipping_name, shipping_address, created_at	Order header, created with status 'placed'. Cascades on deletion of the user.
order_items	id, order_id, product_id, product_name, quantity, price	Order lines. Name and price are denormalised on purpose to snapshot the purchase. No cascade from products, so a product that appears in a historical order cannot be deleted.

4.9.3 REST API Endpoints

Method and path	Auth	Description
POST /api/auth/register	No	Create an account and start a session.
POST /api/auth/login	No	Authenticate and start a session.
POST /api/auth/logout	No	Destroy the session and clear the cookie.
GET /api/auth/me	No	Return the signed-in user, or null.
GET /api/products	No	List products, optionally filtered by search term and category.
GET /api/products/categories	No	Return the distinct category list.
GET /api/products/:id	No	Return a single product.
GET /api/cart	Yes	Return the user's cart lines with a computed total.
POST /api/cart	Yes	Add a product, or increment it if already present.
PUT /api/cart/:id	Yes	Change the quantity of one cart line.
DELETE /api/cart/:id	Yes	Remove one cart line.
POST /api/orders	Yes	Checkout: validate stock and place the order transactionally.
GET /api/orders	Yes	List the user's orders, newest first.
GET /api/orders/:id	Yes	Return one order with its line items.

4.9.4 Project Structure and Development Environment

The repository separates the client from the server: public/ holds the six HTML pages, the stylesheet, the page scripts and the product images, while server/ holds the Express entry point, the database module, the seed script, the authentication middleware and the four route modules. Configuration is read from an environment file that is excluded from version control, with a committed example file documenting the two required keys.

Figure 4.1 shows the project open in Visual Studio Code. The Explorer pane on the left makes the two-folder layout visible: public/ with the six pages and their stylesheet, scripts and images, and server/ with the entry point, the database and seed modules, the authentication middleware and the route files. The file open in the editor is the shared client-side API helper through which every page talks to the server, and the status bar shows the Git branch, the active language and the formatting extension used during development.

4.10 Task 2 - Social Media Platform (CampusLoop)

Overview. CampusLoop is a campus-oriented social network built to facilitate student discussion, academic help, event announcements and opportunity sharing. A student registers with a username, e-mail, password, course and year of study, and lands on a three-column feed. From there the student can publish a text post tagged with one of four categories—Discussion, Study, Event or Opportunity—and optionally mark it as anonymous or as needing help. Other students can like the post, open a chat-style comment thread, and follow or unfollow the author; authors can delete their own posts. A live search box filters the feed instantly by content, category or author.

4.10.1 Key Features Implemented

- Student accounts that also record the course and year of study, with unique usernames and e-mail addresses and a session-based sign-in.
- Categorised posting under Discussion, Study, Event or Opportunity, with optional Anonymous and Help Needed flags that change how the post is displayed.

- Likes and comments on every post, with the comment thread loaded only when the student opens it and shown as chat-style bubbles.
- Follow and unfollow between students, and deletion of one's own posts, which is checked on the server rather than only hidden in the interface.
- Feed with live search that filters the loaded posts by content, category or author as the student types, alongside a trending-topics panel.
- Three-column responsive layout with a profile sidebar, letter-based avatars and toast-style feedback messages.

4.10.2 Database Design

Table	Columns	Purpose and relationships
users	id, username (unique), email (unique), password_hash, course, year, bio, created_at	Student accounts, including the academic details shown in the sidebar profile widget.
posts	id, user_id, content, category, is_anonymous, is_help_needed, created_at	Feed entries. Cascades on deletion of the author.
comments	id, post_id, user_id, content, is_helpful_answer, created_at	Replies to a post, ordered chronologically. Cascades on deletion of either the post or the author.
likes	user_id, post_id composite primary key	Many-to-many link between students and posts; the composite key prevents duplicate likes.
follows	follower_id, following_id composite primary key	Directed many-to-many link between students; the composite key prevents duplicate follows.

4.10.3 REST API Endpoints

Method and path	Auth	Description
POST /api/auth/register	No	Create an account (username, e-mail, password, course, year) and sign in.
POST /api/auth/login	No	Authenticate and start a session.
POST /api/auth/logout	No	Destroy the session.
GET /api/auth/me	Yes	Return the signed-in student's profile.
GET /api/posts	No	Return the feed, newest first, with counts and per-viewer flags.
POST /api/posts	Yes	Publish a post with a category and the two optional flags.
DELETE /api/posts/:id	Yes	Delete a post; rejected with 403 unless the caller owns it.
POST /api/posts/:id/like	Yes	Toggle a like on a post.
GET /api/posts/:id/comments	No	List the comments on a post, oldest first.
POST /api/posts/:id/comments	Yes	Add a comment to a post.
POST /api/users/:id/follow	Yes	Toggle following another student.
GET /api/health	No	Service health check.

4.10.4 Project Structure and Development Environment

CampusLoop uses the same two-folder layout, but with a smaller surface: public/ holds two pages—the authentication screen and the feed—together with one stylesheet and two scripts, while server/ holds the Express entry point, the database module and the SQLite file. Because

the whole application is only two screens, the twelve endpoints are defined inline in the entry point instead of being split into routers.

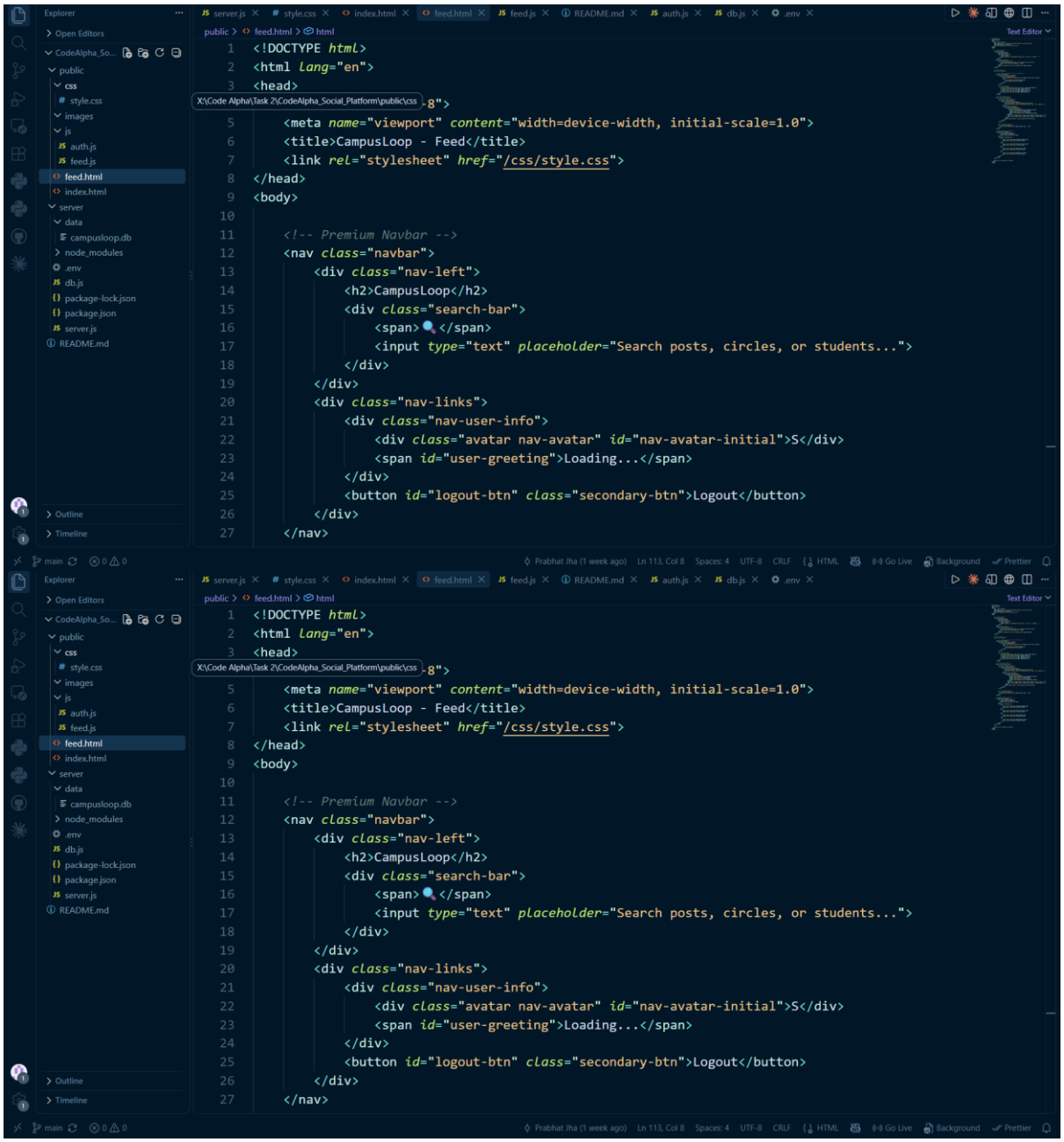

Figure 4.2 shows the project open in Visual Studio Code. The Explorer pane lists the whole project—the stylesheet and the two client scripts, the two pages, and on the server side the database module, the entry point and the SQLite database file. The row of open tabs reflects the full-stack nature of the work, since a single feature such as liking a post touches the schema, the API, the client script and the stylesheet together. The file in the editor is the feed page, the main application shell whose placeholder elements are filled in from the API once the session has been confirmed.

4.11 Results and Outcomes

Both assigned tasks were completed, tested and submitted within the internship period, and both applications run end to end from a clean checkout. The concrete outcomes of the internship are summarised below.

- Two complete, working full-stack web applications were delivered: a transactional e-commerce store with 14 REST endpoints across four resource routers, and a social platform with 12 REST endpoints, each backed by a five-table relational schema.
- Practical ability to design a normalised relational schema and to choose constraints deliberately—unique constraints to prevent duplicate cart lines, composite primary keys to prevent duplicate likes and follows, and cascade rules so that deleting a parent record cleans up its children automatically.
- A working understanding of authentication: why passwords are salted and hashed rather than encrypted, why the session cookie carries only an opaque identifier, why HttpOnly matters, and why login errors should be deliberately vague.
- First-hand experience of database transactions and why they are not optional in a transactional application—the checkout flow was the clearest lesson of the internship in reasoning about partial failure.
- The habit of writing every query as a parameterised prepared statement, and of escaping every value before writing it into the DOM.
- Confidence with the Fetch API, async/await, event delegation and cookie-based session handling on the client, including deliberate handling of loading, empty and error states rather than only the happy path.
- Experience of query design for read performance—specifically, replacing a per-post loop with a single aggregating statement in the feed endpoint.
- Working practice with Git and GitHub, meaningful commit history, README documentation, and keeping configuration and secrets out of version control through environment files.
- Successful completion of the internship, certified by CodeAlpha on 25th August 2026 against Student ID CA/DF1/194729.

Beyond the individual techniques, the most valuable outcome was learning to work across the whole stack for a single feature. Adding something as small as a like button meant touching the schema, the API, the query that reports its state back to the viewer, the client script and the stylesheet—and being responsible for all of them. That end-to-end ownership is what the two tasks were designed to teach, and it is the part of the internship that changed how I approach a problem.

4.12 Certificate of Completion and Proof of Work

The Internship Offer Letter dated 7th July 2026 and the Certificate of Completion dated 25th August 2026, both issued by CodeAlpha against Student ID CA/DF1/194729 and confirming the Full Stack Development Internship carried out from 10th July 2026 to 10th August 2026, are attached in Chapter 3 of this report. Screenshots of the actual development environment for both submitted tasks are included as Figure 4.1 and Figure 4.2 in Sections 4.9.4 and 4.10.4 respectively, as proof of work. The complete source code of both projects, together with their README documentation, was published to GitHub as required by the internship guidelines.

5. BIBLIOGRAPHY/REFERENCES

1. CodeAlpha, "Internship Offer Letter - Full Stack Development Internship," issued 7th July 2026, Student ID: CA/DF1/194729.

2. CodeAlpha, "Certificate of Completion - Virtual Internship Program in Full Stack Development," issued 25th August 2026, Student ID: CA/DF1/194729.

3. Node.js Documentation, https://nodejs.org/docs/latest/api/

4. Express.js Documentation, https://expressjs.com/

5. SQLite Documentation, https://www.sqlite.org/docs.html

6. better-sqlite3 API Documentation, https://github.com/WiseLibs/better-sqlite3/blob/master/docs/api.md

7. express-session Documentation, https://github.com/expressjs/session

8. bcrypt.js Documentation, https://github.com/dcodeIO/bcrypt.js

9. MDN Web Docs - Fetch API, https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

10. MDN Web Docs - CSS Grid Layout, https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout

11. OWASP Foundation, "Password Storage Cheat Sheet," https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

12. Git Documentation, https://git-scm.com/doc